

Day 30 — Sun 4 Oct

Countdown: 6 days to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

🎓 Degree — codegen & optimisation

Source: ComputerScience_Degree/CSD_02_SystemSoftwareAndCompilers.md

14.2 The standard machine-independent optimisations

Optimisation	What it does	Example (before → after)
Constant folding	Evaluate constant expressions at compile time	<code>x = 3 * 4</code> → <code>x = 12</code>
Constant propagation	Replace a variable known to be constant by that constant	<code>a=5; b=a+2</code> → <code>b=7</code>
Common subexpression elimination (CSE)	Compute a repeated expression once	<code>t1=b*c; ...; t2=b*c</code> → reuse <code>t1</code>
Dead-code elimination	Remove code whose result is never used	remove <code>x = y+1</code> if <code>x</code> is never read
Copy propagation	After <code>x=y</code> , use <code>y</code> in place of <code>x</code>	enables more dead-code removal
Strength reduction	Replace an expensive op with a cheaper one	<code>x*2</code> → <code>x+x</code> or <code>x<<1</code> ; <code>i*4</code> in a loop → add 4
Algebraic simplification	Use identities	<code>x+0</code> → <code>x</code> , <code>x*1</code> → <code>x</code> , <code>x*0</code> → <code>0</code>
Loop-invariant code motion	Move computations that don't change in the loop out of it	hoist <code>y = a*b</code> out of a loop over <code>i</code>
Loop unrolling	Replicate the body to cut loop-control overhead	
Induction-variable elimination	Simplify variables that change linearly with the loop counter	

14.4 Peephole optimisation

A **machine-dependent** local optimisation that slides a small "window" (a few instructions) over the target code and replaces recognised inefficient patterns:

Pattern removed	Example
Redundant load/store	<code>STORE R, a</code> then <code>LOAD a, R</code> → drop the load
Unreachable code	code after an unconditional jump with no label
Jumps to jumps	<code>goto L1; ... L1: goto L2</code> → <code>goto L2</code>
Algebraic/strength on target	<code>MUL R, 2</code> → <code>SHL R, 1</code>

🔍 Forensic — hardware & file systems

Source: ComputerForensic/FOR_02_SystemArtifacts.md

1.1 Concept — why an examiner cares about hardware

An ordinary IT technician looks at hardware and asks *"does it work?"*. A forensic examiner looks at the same hardware and asks three completely different questions:

1. **Where can data hide in this thing?** (Every chip that retains state is a potential exhibit — not just the hard disk, but RAM, the BIOS/UEFI NVRAM chip, the SSD's over-provisioned area, the printer's internal memory, the router's flash, the GPU's VRAM.)
2. **How fast will that data disappear if I do the wrong thing?** (The **order of volatility** — see FOR_03 §4.3. Pulling power destroys RAM; booting the machine destroys timestamps.)
3. **How do I get a bit-for-bit copy out without changing a single bit?** (Write blockers, interfaces, adapters, encryption.)

Everything in this section should be read through those three questions.

6.1 File system layout — the Filesystem Hierarchy Standard

Linux presents **one single tree rooted at /**; every device is *mounted* somewhere inside it (there are no drive letters). Learn the directories and, for each, **the forensic reason you would go there**.

Directory	Purpose	Forensic interest
/	Root of everything	—
/bin, /sbin, /usr/bin, /usr/sbin	Essential and system binaries	Trojaned system binaries (a classic rootkit) — verify against package hashes (<code>rpm -Va</code> , <code>debsums</code>)
/boot	Kernel (<code>vmlinuz</code>), <code>initrd</code> , GRUB config	Bootkits; <code>grub.cfg</code> shows boot parameters
/dev	Device nodes	/dev/sda, /dev/sda1 naming; /dev/null, /dev/random
/etc	All system configuration	<code>passwd</code> , <code>shadow</code> , <code>group</code> , <code>sudoers</code> , <code>hosts</code> , <code>hostname</code> , <code>fstab</code> , <code>crontab</code> , <code>resolv.conf</code> , <code>ssh/</code> , <code>network/</code> . The first place to look
/home/<user>	User home directories	User data, dot-files (§6.4), <code>.bash_history</code> , <code>.ssh/</code>
/lib, /lib64, /usr/lib	Shared libraries	LD_PRELOAD rootkits, malicious <code>.so</code> files
/media, /mnt	Mount points for removable and temporary media	Evidence that a USB device was mounted
/opt	Third-party software	Non-package-managed (often unauthorised) applications
/proc	Virtual file system exposing kernel/process state — exists only in RAM	On a live system: <code>/proc/<pid>/cmdline</code> , <code>/exe</code> , <code>/fd</code> , <code>/maps</code> , <code>/net/tcp</code> . Nothing here appears in a dead-box image — a key live-forensics point

<code>/root</code>	The root user's home	Root's <code>.bash_history</code> — often the whole intrusion story
<code>/sys</code>	Virtual kernel/device interface	Live only
<code>/tmp</code> , <code>/var/tmp</code>	Temporary files	Attackers' favourite working directory (world-writable). <code>/tmp</code> is often cleared at boot or is a tmpfs (RAM) — <code>/var/tmp</code> survives reboot
<code>/usr</code>	User programs, docs, source	<code>/usr/local/</code> for locally compiled binaries
<code>/var</code>	Variable data	<code>/var/log (\$6.6)</code> , <code>/var/spool/{mail,cron,cups}</code> , <code>/var/www</code> (web content and upload directories — webshells), <code>/var/lib</code> (databases, package state, Docker)